

shortn — AWS Infrastructure Setup

Module 3, Stage 5 · ElastiCache + EC2 + RDS on a custom VPC

Resource	Name	Purpose
VPC	project-vpc (10.0.0.0/16)	Isolated network for all resources
Public Subnets (x2)	project-subnet-public1/2	EC2 app server — internet-facing
Private Subnets (x2)	project-subnet-private1/2	ElastiCache + RDS — internal only
EC2	shortn-web-server (t3.micro)	Node.js app server
ElastiCache	shortn-cache (cache.t3.micro)	Redis cache for URL lookups
RDS	shortn-db (db.t3.micro)	Postgres primary database
SG: shortn-sg-web	EC2 security group	Inbound: SSH:22, TCP:3000 from your IP
SG: shortn-sg-cache	ElastiCache security group	Inbound: TCP:6379 from shortn-sg-web
SG: shortn-sg-db	RDS security group	Inbound: TCP:5432 from shortn-sg-web

Step 1 — Create the VPC

A VPC is your private isolated network in AWS. All resources (EC2, ElastiCache, RDS) live inside it and can talk to each other without going through the public internet.

Console: VPC → Create VPC → *VPC and more*

Setting	Value	Why
Name tag	project	Auto-names all child resources
IPv4 CIDR	10.0.0.0/16	65,536 addresses — room for 4 subnets
AZs	2	Required for Multi-AZ ElastiCache + RDS
Public subnets	2	EC2 needs internet access
Private subnets	2	ElastiCache + RDS — never public
NAT Gateway	None	Saves ~\$32/month — EC2 is in public subnet
VPC Endpoints	None	Not needed for this setup

VPC itself is free. Subnets, route tables, and internet gateway are also free. Only compute resources (EC2, RDS, ElastiCache) cost money.

Step 2 — Create Security Groups

Security groups are stateful firewalls. Create three — one per resource type. The key rule: ElastiCache and RDS never accept connections from the internet, only from the EC2 security group.

Naming convention: <project>-sg-<role>

Security Group	Attached To	Inbound Rule	Purpose
shortn-sg-web	EC2	SSH:22 + TCP:3000 from your IP	App server access
shortn-sg-cache	ElastiCache	TCP:6379 from shortn-sg-web SG	Only EC2 can reach Redis
shortn-sg-db	RDS	TCP:5432 from shortn-sg-web SG	Only EC2 can reach Postgres

Source = SG reference (not CIDR). This means rules auto-update if the EC2 IP changes — you never need to update port rules manually.

Step 3 — Launch EC2 Instance

Console: EC2 → Launch Instance

Setting	Value
AMI	Amazon Linux 2023
Instance type	t3.micro
Key pair	terminal_keys (existing)
VPC	project-vpc
Subnet	project-subnet-public1-us-east-1a (public)
Auto-assign public IP	Enable
Security group	shortn-sg-web

EC2 goes in a public subnet so you can SSH in directly. ElastiCache and RDS go in private subnets — only reachable from inside the VPC.

Step 4 — Create ElastiCache Redis

4a — Create Subnet Group

Console: ElastiCache → Subnet Groups → Create subnet group

Name: **shortn-cache-subnet-group** · VPC: project-vpc · Add both private subnets (us-east-1a + us-east-1b)

Subnet group tells ElastiCache which subnets it's allowed to place nodes in. Always use private subnets.

4b — Create Redis Cluster

Console: ElastiCache → Redis OSS caches → Create Redis OSS cache

Setting	Value	Why
Engine	Redis OSS 7.1	Standard Redis — Valkey is newer but Redis is fine for learning
Deployment	Node-based cluster	Gives control over node type and count

Creation method	Cluster cache	Full configuration control
Cluster mode	Disabled	Single shard — data fits on one node for shortn
Multi-AZ	Disabled (single node)	Re-enable when adding replica
Node type	cache.t3.micro	0.5 GiB — sufficient for URL cache
Replicas	0 (single node first)	Add replica in next step for Multi-AZ
Subnet group	shortn-cache-subnet-group	Private subnets only
Security group	shortn-sg-cache	Only EC2 can connect on 6379
Encryption in transit	Enabled (TLS required)	Use rediss:// in connection string
Backups	Disabled	Not needed for learning — saves cost

Step 5 — Create RDS Postgres

5a — Create DB Subnet Group

Console: RDS → Subnet groups → Create DB subnet group

Name: **shortn-db-subnet-group** · VPC: project-vpc · Add both private subnets

5b — Create Database

Console: RDS → Create database

Setting	Value
Engine	PostgreSQL (free tier)
DB identifier	shortn-db
Master username	postgres
VPC	project-vpc
DB subnet group	shortn-db-subnet-group
Public access	No
Security group	shortn-sg-db
RDS Proxy	Disabled (extra cost, not needed)

Step 6 — Deploy App on EC2

SSH into EC2 and run:

```
sudo yum install -y git nodejs npm redis6
git clone <your-repo>
cd shortn && npm install
```

Set environment variables:

```
export REDIS_URL=rediss://master.shortn-cache.by45x.use1.cache.amazonaws.com:6379
export DATABASE_URL=postgres://postgres:<password>@shortn-db.cbqmqa0ummk8.us-east-1.rds.amazonaws.com:5432/postgres
```

Note rediss:// with double-s — TLS is required. Single redis:// will be refused by ElastiCache.

In your ioredis client config, add explicit TLS option:

```
const redis = new Redis(process.env.REDIS_URL!, {
  tls: {},
  commandTimeout: 500, // calibrated to p99 under load, not local baseline
});
```

Key Learnings from This Setup

Lesson	Detail
VPC is free to keep	Only compute resources (EC2, RDS, ElastiCache) cost money. Keep the VPC between sessions, tear down compute resources.
Never put cache/DB in public subnets	ElastiCache and RDS have no public endpoints. Security groups enforce this at the network layer.
SG-to-SG reference > CIDR	Referencing a security group as a source auto-updates if IPs change. CIDRs go stale.
commandTimeout != local Redis latency	ElastiCache under 1000 VUs spiked past 100ms. Calibrate timeout to 3-5x p99 under peak load, not avg at idle.
rediss:// not redis://	Encryption in transit is enforced by ElastiCache. Double-s = TLS. Single-s = connection refused.
Cross-AZ latency is real	Local Redis: ~0.1ms. ElastiCache same-AZ: ~0.5ms avg. This changes your circuit breaker calibration.

Teardown Checklist (run at end of every session)

VPC, subnets, route tables, and security groups can stay — they are free.

- [] ElastiCache: delete shortn-cache cluster
- [] RDS: delete shortn-db instance (skip final snapshot)
- [] EC2: terminate shortn-web-server instance
- [] EC2: release any Elastic IPs if allocated
- [] Verify next morning in Cost Explorer: zero new charges